

チュートリアル

1章 言語の作成理由

言語の名前はFermaQED。この言語を作成する理由として、一つの関数で全ての変数の中身を確認できることを目的に作成した。フェルマとは、数学者の名前として有名な人物である。そのため、FermaQEDでは、扱える値は数値のみに制限している。

そして、簡易的な言語処理系であると共にプログラミング初心者が終端文字、基本的な制御文の使い方、配列を元にした辞書の使い方を直感的に理解できることを目指して作成した。

2章 自作言語の構文式

生成規則

- ・ 文列 = 文 (文) *
- ・ 文 = 配列変数 `=` `式` QED | if 関数 `:` `文` (elif 関数 `:` `文`)* (else `:` `文`) ? QED | while 関数 `:` `文` QED | print `(` `式`)` QED | Ferma `(` ` ` )` QED
- ・ 関数 = 大比較 `(` `式` , `式`)` | 小比較 `(` `式` , `式` )` | 等しい `(` `式` , `式`)`
- ・ 式 = 項 (`(` `+` `|` `-` `)` 項) *
- ・ 項 = 因子 (`(` `\*` `|` `/` `)` 因子) *
- ・ 因子 = リテラル | 配列変数 | `(` `式`)`
- ・ 配列変数 = `[` `リテラル` `]`

3章 使い方

①count.fermaに以下のコードを入力する

```
...  
[0] = 3QED  
Ferma()QED  
...
```

②コマンドプロンプトに以下のコマンドを入力する

```
...  
% ruby FermaQED.rb count.ferma  
...
```

③出力

```
...  
{ "0" => 3 }  
...
```

4章 配列変数と終端文字について

①配列変数

配列変数とは、普通の変数と同じく扱うことができる。しかし、変数の名前に指定できるのは扱える値は数値のみであり文字の名前をつけられないことに注意が必要。

②終端文字

終端文字とは、1つの文の切れ目を明確にする記号である。

基本的には、制御構文、関数 (組み込み関数) の後などにつけるものである。

FermaQED における終端文字 QED は、Java におけるセミコロン (;) と同様の役割を果たす。

```
...  
[0] = 1QED  
...  
...  
if equal_comparison(1,1):  
    print(6)QED  
QED  
...  
...  
    print(1+1)QED  
...
```

5章 式について

式とは、数値の計算を行うための構文であり、FermaQED では四則演算（加算、減算、乗算、除算）に対応している。式には項が1つのみで演算子がなくても動かすことができる。

```
...  
1+1QED  
...  
...  
2-2QED  
...
```

6章 制御構文について

①代入文

代入文は配列変数に代入するための構文。

```
...  
[0] = 3QED  
...
```

②if文

if文は比較関数(条件式)を元にどの文を実行するかを判定する。基本的には、他のプログラミング言語のif文と同じくtrueになった時点でその文が実行され、以降の条件の比較は行わない。

全ての条件式に合わない時はelseが実行される。

```
...  
if large_comparison(2,1):  
    print(1)QED  
QED  
...  
...  
if large_comparison(2,2):  
    print(1)QED  
elif large_comparison(2,1):  
    print(2)QED  
QED  
...
```

```

...
    if large_comparison(2,2):
        print(1)QED
    else :
        print(2)QED
QED
...

```

③while文

while文は比較関数(条件式)がtrueの時のみ文を無限に処理する。比較関数の値には、配列変数を指定しないと無限ループになるので注意が必要

```

...
    [5] = 10QED
    print([5])QED
    while large_comparison([5],1):
        [ 5 ]= [5] - 1QED
    QED
    Ferma()QED
...

```

7章 比較関数(条件式)について

if文とwhile文の:前におくことになる条件分岐を行うために必要な関数。

比較関数には終端文字であるQEDがつかない。

①左が右の値よりも多い時にtrueを返す

```

...
    large_comparison(5,1)
...

```

②左が右の値よりも小さい時にtrueを返す

```

...
    small_comparison(1,2)
...

```

③左と右の値が等しい時にtrueを返す

```

...
    equal_comparison(1,1)
...

```

8章 関数(組み込み関数)について

FermaQEDにすでに用意された機能呼び出すための関数。現在、2つの関数のみが搭載されている。

① print()

計算式の結果や指定した配列変数の中身を確認するための関数。

```

...
    print(6)QED
...

```

②Ferma()

配列変数全ての値を一度に出力するための関数。コードのデバッグ向けの用途向け。

```

...

```

```
Ferma()QED
...
```

9章 サンプルプログラムについて

①代入文、式と関数(組み込み関数)のチュートリアル

```
```コード(samp1.ferma)
```

```
[0] = 1QED
[1] = [0]QED
```

```
[2] = 2+1QED
```

```
print(2+2)QED
print([2])QED
```

```
Ferma()QED
...
```

```
```出力
```

```
4
3
{"0"=>1, "1"=>1, "2"=>3}
...
```

②if文とwhile文、比較関数(条件式)のチュートリアル

```
```コード(samp2.ferma)
```

```
if large_comparison(2,1):
 print(1)QED
QED
```

```
if small_comparison(1,2):
 print(2)QED
QED
```

```
if equal_comparison(1,1):
 print(3)QED
QED
```

```
[5] = 10QED
print([5])QED
while large_comparison([5],1):
 [5]= [5] - 1QED
QED
```

```
Ferma()QED
...
```

```
```出力
```

```
1
2
3
10
```

```
{ "5"=>1}  
...
```

③全ての要素を一度に実行するコードのチュートリアル

```
``コード(samp3.ferma)  
1+1QED  
  
[0] = 1QED  
[1] = [0]QED  
[3] = 3QED  
[2] = 1+2QED  
[4] = [0] + [0]QED  
  
print([0])QED  
print(1+1)QED  
  
Ferma()QED  
  
[5] = 10QED  
print([5])QED  
while large_comparison([5],1):  
    [ 5 ]= [5] - 1QED  
QED  
Ferma()QED  
  
if large_comparison(2,1):  
    print(1)QED  
QED  
  
if large_comparison(1,2):  
    print(2)QED  
QED  
  
if small_comparison(1,2):  
    print(3)QED  
QED  
  
if small_comparison(2,1):  
    print(4)QED  
QED  
  
if equal_comparison(2,1):  
    print(5)QED  
QED  
  
if equal_comparison(1,1):  
    print(6)QED  
QED
```

```

if large_comparison(2,2):
    print(7)QED
elif large_comparison(2,1):
    print(8)QED
QED

if large_comparison(2,2):
    print(9)QED
elif large_comparison(2,1):
    print(10)QED
elif large_comparison(2,1):
    print(11)QED
QED

if large_comparison(2,2):
    print(11)QED
elif large_comparison(2,1):
    print(12)QED
elif large_comparison(2,1):
    print(13)QED
QED

if large_comparison(2,2):
    print(14)QED
elif large_comparison(2,2):
    print(15)QED
elif large_comparison(2,1):
    print(16)QED
QED

if large_comparison(2,2):
    print(17)QED
elif large_comparison(2,2):
    print(18)QED
elif large_comparison(2,2):
    print(19)QED
else :
    print(20)QED
QED

if large_comparison(2,1):
    if large_comparison(2,1):
        print(21)QED
    QED
QED

Ferma()QED
` ``

```

````出力

1

2

{"0"=>1, "1"=>1, "3"=>3, "2"=>3, "4"=>2}

10

{"0"=>1, "1"=>1, "3"=>3, "2"=>3, "4"=>2, "5"=>1}

1

3

6

8

10

12

16

20

21

{"0"=>1, "1"=>1, "3"=>3, "2"=>3, "4"=>2, "5"=>1}

````